

Examen Parcial de IA

(31 de Octubre de 2025)

Duración: 110 min.

1. (6 puntos) Dado el interés creciente que despierta el juego del GO, la junta directiva de la Asociación de GO necesita establecer un calendario para la final del campeonato anual de GO, que este año tendrá muchos más participantes. En la final del campeonato hay C diferentes categorías dependiendo de la edad y del nivel de los jugadores (30 Kyu .. 1 Kyu, 1 dan .. 9 dan). En cada categoría participan un número J_c de jugadores, lo cual significa un número R_c de rondas eliminatorias (obviamente $R_c = \log_2(J_c)$). La final ha de durar un máximo de 15 días, para cada día de la final se disponen de F franjas horarias, en una franja solo se puede jugar una partida de GO. Obviamente se han de jugar todas las partidas de la final del campeonato.

El calendario ha de cumplir un conjunto de restricciones:

- El número de días necesarios para realizar la final del campeonato ha de ser el mínimo posible.
- El conjunto de rondas correspondientes a cada categoría se ha de celebrar en el mínimo número de días posible.
- En un día se pueden jugar como máximo P partidas de una categoría.
- Entre dos partidas de una misma categoría en un mismo día debe haber como mínimo dos franjas horarias de separación y como máximo cinco.
- Debe haber como mínimo un día de separación entre dos rondas de una categoría.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (estado o solución inicial, operadores, función heurística,...). Para cada una de ellas se ha de comentar los diferentes elementos de la propuesta, analizando si la técnica escogida es adecuada para este problema, si cada uno sus elementos son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

Anotación didáctica inicial En este enunciado era importante entender bien la relación entre categorías, rondas y partidas para poder asociarlo correctamente a franjas y días.

En algunas respuestas se han confundido partidas y rondas, como si fuera lo mismo. Algunos alumnos/as han asumido que el número total de partidas es proporcional al número de rondas por categoría ($PT = C * R_c$), pero esto no es correcto.

Una recomendación para cuando tengáis dudas sobre como se relacionan elementos del enunciado es haceros pequeños ejemplos. Por ejemplo, si suponemos que tenemos $J_c = 8$ participantes en una categoría:

- en la primera ronda hay $J_c/2 = 4$ partidas
- en la segunda ronda hay $J_c/4 = 2$ partidas
- en la tercera ronda hay $J_c/8 = 1$ partida
- en total tenemos $J_c = 8$, $P_c = 4 + 2 + 1 = 7$, $R_c = 3$ (aquí ya vemos que no tenemos el mismo número de rondas que de partidas).

Si suponemos ahora que tenemos $J_c = 16$ participantes en una categoría:

- en la primera ronda hay $J_c/2 = 8$ partidas
- en la segunda ronda hay $J_c/4 = 4$ partidas
- en la tercera ronda hay $J_c/8 = 2$ partidas
- en la cuarta ronda hay $J_c/16 = 1$ partida
- en total tenemos $J_c = 16$, $P_c = 8 + 4 + 2 + 1 = 15$, $R_c = 4$.

Generalizando esto a cualquier número J_c de participantes en una categoría:

- $R_c = \log_2(J_c)$,

- $J_c = 2^{R_c} = P_c + 1$
- $P_c = J_c - 1 = 2^{R_c} - 1$ (aquí ya vemos que no hay relación lineal entre el número de partidas y el número de rondas)

En los apartados siguientes, par calcular costes y factores de ramificación, es útil tener el número total de partidas (PT) y el número total de franjas (FT). Como cada categoría puede tener más o menos jugadores, y más o menos rondas, no es correcto estimar (PT) a partir de una J_c o R_c concreta. Lo mejor es hacer el sumatorio para todas las categorías:

$$JT = \sum_{c=1}^C J_c, \quad RT = \sum_{c=1}^C R_c, \quad PT = \sum_{c=1}^C P_c, \quad FT = 15 * F, \quad DT = 15$$

Ahora que ya tenemos todos los valores totales, podemos expresar mejor los costes y factores de ramificación en nuestras respuestas.

- a) Usar Hill-climbing. Como solución inicial se utiliza el calendario vacío. Como operadores usamos asignar y desasignar una partida de una ronda de una categoría a una franja de un día. Como función heurística usamos el número total de días en los que hay partidas asignadas más la suma del número de días que ocupan las rondas de una categoría.

A continuación se muestra una posible solución, puntuada por apartados (sobre 2,0 puntos más extras). Más abajo se muestra un razonamiento más completo sobre el problema, como anotaciones didácticas.

- 0.2 puntos Hill-climbing es una técnica adecuada para este problema, ya que se requiere minimizar días. Eso si, no asegura encontrar el óptimo.
- 0.2 puntos La solución inicial no cumple la condición de que se asignen todas las partidas en el calendario, por lo que no es solución. Su calidad es pésima (nos piden asignar correctamente todas las partidas, pero no tenemos ninguna asignada). Se podría considerar un planteamiento correcto si aseguráramos que durante la búsqueda llegaremos a una solución válida.
- 0.1 punto El coste de generación es nulo ($O(1)$).
- 0.2 puntos Los operadores de asignar y desasignar permiten obtener todas las soluciones, dándonos una cobertura total del espacio de soluciones.
- 0.2 puntos El operador asignar debería comprobar las restricciones tercera, cuarta (más de 2 y menos de 5 franjas) y quinta que se imponen para ser solución,
- 0.1 punto el factor de ramificación del operador asignar en el caso peor es $O(PT \cdot FT)$, siendo PT el número total de partidas del campeonato y FT el número total de franjas del calendario, ya que para cada partida podemos intentar asignarla a cualquiera de las franjas del calendario que no esté ocupada.
- 0.1 punto El operador desasignar debería comprobar la restricción cuarta (menos de 5 franjas) ya que al eliminar una partida de una categoría se podría producir un espacio mayor de 5 franjas entre la partida anterior y la partida siguiente de esa categoría,
- 0.1 punto el factor de ramificación del operador desasignar es solo $O(PT)$, ya que para cada partida solo hemos de desasignarla.
- 0.1 punto La función heurística se ha de minimizar para cumplir los objetivos del problema.
- 0.2 puntos La función heurística incluye los dos criterios principales a optimizar, pero no es demasiado informativa y no será capaz de distinguir entre las soluciones vecinas, ya que al insertar o eliminar una sola partida que muchos vecinos tendrán el mismo valor del heurístico (días totales). Por lo tanto es una función con muchas mesetas, cosa que va a bloquear fácilmente la exploración del Hill-climbing.
- 0.1 punto En los casos en los que los operadores si modifican el número de días, con el heurístico actual los sucesores del operador asignar *nunca* se escogerán, solo los del operador deasignar, ya que el objetivo es minimizar días.
- 0.2 puntos De hecho, la solución inicial vacía ya nos da el valor mínimo del heurístico, y por lo tanto esta combinación de estado inicial, operadores y heurístico con este algoritmo no hará **nada**.

0.1 punto Para guiar correctamente el algoritmo el heurístico debería penalizar los casos en los que se incumplen alguna de las restricciones o que falten partidas por colocar.

0.1 punto También estaría bien añadir pesos a cada criterio para ajustar su relevancia.

Puntos extra:

0.1 punto Como es muy difícil encontrar en Hill Climbing heurísticos que funcionen bien con operadores de construcción y destrucción a la vez (en este caso, asignar y desasignar), sería mejor plantear, a partir de una solución vacía, el operador asignar acompañado de un operador mover o un operador intercambiar.

0.2 puntos El enunciado no describe explícitamente la representación de una solución (y por ello el análisis de la representación no entra en la puntuación básica de la respuesta), pero como los operadores asignan y desasignan partidas a franjas horarias, lo más adecuado sería un vector de $F * 15$ posiciones donde a cada una se puede asociar un identificador de partida. Suponiendo que cada partida tiene un identificador numérico de partida codificado en binario, el coste espacial de esta representación sería $\lceil \log_2 PT \rceil \cdot FT$.

Anotación didáctica el objetivo de la solución planteada es mostrar el problema que suele surgir cuando se tiene un heurístico que se limita a cubrir solo los criterios a minimizar/maximizar. En este caso el heurístico no es capaz de distinguir pequeñas modificaciones en la colocación de las partidas si no modifican el número total de días necesarios por categoría o para todo el torneo, por lo que no será capaz de guiar el uso de los operadores de asignar y desasignar. Tenemos, por lo tanto, un heurístico con demasiadas mesetas que va a bloquear enseguida la exploración del Hill Climbing. La adición de penalizaciones graduadas por cada tipo de no-solución (es decir, penalizaciones proporcionales al número de partidas pendientes de colocar, al número de veces que el orden no es adecuado, al número de franjas de distancia entre dos partidas consecutivas de la misma categoría fuera de rango entre 2 y 5, etc...) puede reducir las mesetas dentro del espacio de no-soluciones (dando valores diferentes entre dos no-soluciones que coincidan en número de días), pero seguiríamos teniendo mesetas en el espacio de soluciones.

Tenemos también en la solución propuesta la dificultad de plantear un heurístico que pueda guiar adecuadamente a un Hill Climbing cuando se tienen a la vez operadores de construcción (asignar) y destrucción (desasignar). De nuevo el añadir penalizaciones graduadas podría guiar la aplicación de ambos operadores (por ejemplo asignar reduce el número de partidas sin asignación, desasignar puede reducir el número de partidas asignadas en orden incorrecto) pero requeriría buscar un complejo equilibrio de pesos entre cada uno de los penalizadores dentro de la función.

En esta solución se combinaba un mal heurístico con una mala solución inicial. Empezar un Hill-climbing con una no-solución de tan mala calidad no es una buena idea cuando es viable generar una solución inicial (o una no-solución mejor) con un coste de generación bajo, ya que el algoritmo se pasará la mayor parte del tiempo intentando generar una solución, en vez de intentando mejorar una solución inicial. Además este heurístico era pésimo para la solución inicial vacía, ya que coincidía con el mínimo de la función.

Algunos alumnos/as han afirmado que en Hill-climbing es incorrecto empezar con una no-solución inicial, que siempre la solución inicial ha de ser solución. Esto es falso y un error conceptual. En Hill-climbing es **preferible** (no obligatorio) que la solución inicial esté en el espacio de soluciones, pero si el coste de empezar en una solución es demasiado elevado, se busca un generador que genere una no-solución de la mejor calidad posible dentro de un coste de generación aceptable.

Algunos alumnos/as han sugerido penalizar con coste infinito las no-soluciones. Esto es muy mala idea en Hill-climbing, ya que crea una meseta en todo el espacio de no-soluciones. Se necesitan penalizadores proporcionales a cada uno de los tipos de no-soluciones para que puedan distinguir entre una no-solución mala y una peor, y guiar así al algoritmo desde el espacio de no-soluciones al espacio de soluciones.

- b) Usar Hill-climbing. Como solución inicial se usa el calendario resultante de añadir de manera secuencial las partidas de cada ronda de la primera categoría manteniendo una distancia de cinco franjas horarias entre cada partida, sin superar las P partidas diarias y manteniendo un día de separación entre cada ronda. Con el mismo procedimiento se van introduciendo las partidas del resto de categorías. Como operadores utilizamos mover una partida a una franja no ocupada e intercambiar dos partidas. Como función heurística usamos el número de días total en los que hay partidas asignadas, más la suma del

número de días que ocupan las rondas de una categoría, más una constante por el número de franjas que separan en un día las partidas de una misma ronda y categoría.

A continuación se muestra una posible solución, puntuada por apartados (sobre 2,0 puntos más extras). Más abajo se muestra un razonamiento más completo sobre el problema, como anotaciones didácticas.

- 0.2 puntos Como se ha dicho en el apartado anterior, Hill-climbing es una técnica adecuada para este problema, ya que el problema requiere minimizar una serie de criterios. Eso sí, no asegura encontrar el óptimo.
- 0.2 puntos La solución inicial será solución solo si se puede garantizar que imponiendo que entre cada partida haya cinco franjas de separación (que es el máximo que admitimos) quepan todas las partidas en 15 días. De hecho, dependiendo de la representación de la solución, si solo contiene 15 días podría dejar partidas sin asignar.
- 0.1 punto La calidad de esta solución será media-baja, ya que estamos imponiendo al separación máxima en franjas entre partidas de una misma categoría. Dependiendo del número de categorías y de jugadores por categoría es posible que no se puedan rellenar los espacios dejados por una categoría con las otras categorías (sobre todo en las rondas finales, que tienen menos partidos), y que eso haga que se usen más días de los imprescindibles.
- 0.1 punto El coste de generación de la solución inicial en el caso peor es $O(PT \cdot FT)$, ya que para cada categoría hay que colocarse al principio del calendario, y para cada partida hay que hacer una búsqueda en el calendario de la primera franja libre que tenga la distancia adecuada con las otras partidas de la categoría.
- 0.4 puntos Los operadores pueden generar todas las soluciones posibles solo en los casos en los que la solución inicial tiene todas las partidas asignadas. De hecho el operador mover es suficiente para generarlas todas, ya que un intercambio se puede obtener a partir de movimientos (solo hace falta un hueco en el calendario) y en este caso es probable que haya huecos por el generador escogido para la solución inicial. Pero dependiendo del comportamiento de la función heurística, es posible que la exploración solo con mover se vea muy limitada y que se encalle enseguida.
- 0.3 puntos Ambos operadores deberían comprobar que las soluciones generadas cumplan todas las restricciones (tercera, cuarta y quinta) para ser solución.
- 0.1 punto El factor de ramificación de mover depende del producto entre huecos y partidas, en el caso peor es $O(PT \cdot FL)$, siendo PT el número total de partidas del campeonato y FL el número franjas del calendario no ocupadas por partidas ($FL = FT - PT$), ya que para cada partida podemos intentar moverla a cualquiera de las franjas del calendario que no esté ocupada.
- 0.1 punto El factor de ramificación de intercambiar es cuadrático respecto a las partidas: $O(PT^2)$
- 0.2 puntos La función heurística es más informativa que la del apartado anterior ya que intenta minimizar también la distancia entre partidas de la misma ronda y categoría, pudiendo distinguir soluciones con el mismo número de días, orientando la búsqueda hacia asignaciones con las partidas lo mas juntas posibles. Esto puede ayudar a reducir el número total de días. Sin embargo, al no controlar la separación mínima de 2 franjas entre partidas de una misma ronda y categoría puede empujar a no-soluciones.
- 0.1 punto Otro problema del heurístico es que aun tiene mesetas. Por ejemplo: si tenemos 3 partidas de una misma ronda y categoría colocadas en un mismo día, un hueco de 3 franjas entre la primera y la segunda y un hueco de 2 franjas entre la segunda y la tercera (ocupando un total de 8 franjas), si el operador mover intenta acercar la segunda partida a solo 2 franjas de la primera (para luego acercar la tercera a la segunda) el movimiento de acercamiento de la segunda partida va a ser eliminado de los sucesores ya que no modifica ninguno de los criterios del heurístico (las tres partidas siguen ocupando 8 franjas y los mismos días)
- 0.2 puntos Para guiar correctamente el algoritmo el heurístico debería penalizar los casos en los que se incumplen alguna de las restricciones, y añadir pesos para poder equilibrar la influencia de cada una de las penalizaciones por no-solución.

Puntos extra:

- 0.1 punto Una posible mejora en el generador de la solución inicial sería añadir algo de aleatoriedad (como hace, por ejemplo, el generador del siguiente apartado). De esta forma se podría lanzar el Hill-climbing más de una vez y obtener resultados diferentes.

0.2 puntos en el caso de que la solución inicial no tenga todas las partidas asignadas se debería añadir un operador de asignar partidas, y se debería añadir al heurístico un penalizador proporcional al número de partidas no asignadas para guiar la aplicación de este operador.

Anotación didáctica el objetivo de la solución planteada es mostrar que un planteamiento que a priori parece mejor (solución inicial completa, operadores de cambio entre soluciones completas) pueden no serlo tanto, ya que se ha de comprobar que efectivamente la solución inicial lo sea, y que los operadores tengan una cobertura total partiendo de esa solución.

El enunciado no nos dice si en la representación de la solución el calendario tiene 15 días o si puede tener más, y por lo tanto tenemos dos tipos de no-solución inicial (y esto impacta en los operadores):

- Si la representación permite colocar partidas del día 16 en adelante, la solución inicial puede ser no-solución por no cumplir que todos los partidos se celebran en un máximo de 15 días. Pero al tener todas las partidas asignadas, los operadores mover e intercambiar son suficientes para cubrir todo el espacio de soluciones.
- Si la representación solo permite colocar partidas en 15 días, la solución inicial puede ser no-solución por tener partidas sin asignar (las que no han cabido en los primeros 15 días). En este caso los operadores mover e intercambiar no son suficientes para cubrir todo el espacio de soluciones, y es necesario añadir un operador asignar para poder incluir en la solución las partidas sin asignar.

Algunos alumnos/as criticaron que el heurístico incluyera un criterio "que el problema no pide que se minimice" (el número de franjas que separan en un día las partidas de una misma ronda y categoría). Pero esto no es de por sí incorrecto. Cuando se busca un heurístico para guiar los operadores de un Hill-climbing, la primera función candidata es la que incorpora únicamente los criterios que explícitamente se han de maximizar o minimizar, pero a menudo esa función presenta mesetas o no permite que alguno de los operadores se aplique adecuadamente, y por lo tanto se han de añadir elementos extra en la función heurística.

c) Usar Algoritmos Genéticos. Asignamos a cada partida a jugar un código binario con la longitud necesaria para representarlas todas, añadimos a la codificación un código con todos los bits a cero. Representamos un calendario como una cadena de bits de longitud igual al producto de F franjas por 15 días por el número de bits necesarios para representar las partidas. Si una franja no tiene una partida asignada contendrá la codificación a cero, si no tendrá el código de una partida.

Para generar la población inicial usamos el método del segundo apartado cambiando el orden en el que escogemos las categorías para generar variabilidad. Como operadores genéticos usamos solamente el de cruce por un punto, donde el punto de cruce no puede estar en medio de la codificación de una partida. Como función heurística minimizamos el número de franjas vacías entre dos partidas consecutivas.

A continuación se muestra una posible solución, puntuada por apartados (sobre 2,0 puntos más extras). Más abajo se muestra un razonamiento más completo sobre el problema, como anotaciones didácticas.

0.2 puntos AG es una técnica adecuada para este problema ya que nos piden minimizar criterios y en este dominio hay una cierta dificultad para generar heurísticos que no tengan mesetas (las mesetas son problemáticas para HC, pero no para AG). Eso sí, al igual que en los apartados anteriores, no podemos asegurar el encontrar el óptimo.

0.1 punto La representación es correcta, tenemos las asignaciones de partidas a las franjas horarias disponibles, aunque tendremos muchos bits a cero (franjas que no están ocupadas por una partida). Permite codificar todas las soluciones...

0.3 puntos ... pero también permite codificar seis tipos de no-soluciones:

- $NS1$ identificadores de partida inválidos, si $(PT + 1)$ no es exactamente un número potencia de 2 (los $\lceil \log_2 PT + 1 \rceil$ bits codifican más identificadores de partida de los necesarios).
- $NS2$ partidas asignadas a más de una franja,
- $NS3$ partidas no asignadas a alguna franja,
- $NS4$ más de P partidas de una categoría por día,
- $NS5$ menos dos o más de cinco franjas horarias de separación entre dos partidas de una misma categoría en un mismo día,

- *NS6* menos de un día de separación entre dos rondas de una categoría.

El único tipo de no-solución que la representación no permite representar es que el torneo tenga asignadas partidas en más de 15 días.

- 0.2 puntos Salvo lo comentado en el apartado anterior, la generación de soluciones es correcta siempre que la asignación de partidas no supere los 15 días, ya que en este caso las partidas que se deberían asignar del día 16 en adelante quedarían sin asignar.
- 0.1 punto El generador ofrece cierta variabilidad de soluciones iniciales....
- 0.1 punto ...pero esta variabilidad es limitada si tamaño de la población inicial supera los ordenes posibles de las categorías ($C!$), ya que este será el máximo número de soluciones diferentes que se podrán generar.
- 0.1 punto El coste de generación en el caso peor es $O(N \cdot PT \cdot FT)$, donde N es el número de individuos de la generación inicial del algoritmo genético, PT es el número total de partidas. y FT es el número total de franjas del calendario.
- 0.3 puntos el operador de cruce propuesto, a pesar de no cortar dentro de la codificación de los identificadores de partida, puede generar casi todos los tipos de no-soluciones (*NS2*, *NS3*, *NS4*, *NS5* y *NS6*).
- 0.2 puntos En este caso la falta de operador de mutación no solo ralentiza la exploración del espacio de soluciones, sino que se convierte en un serio problema. Como existe la posibilidad de que la solución inicial deje partidas sin asignar y el operador de cruce no puede generar nuevos identificadores de partida, el resultado es que en el caso de que alguna partida no aparezca en ninguna de las soluciones iniciales, AG no encontrará ninguna solución.
- 0.1 punto La función heurística propuesta se ha de minimizar, pero AG siempre maximiza la función que se le pasa (ya que busca el *fitness* máximo de las soluciones al problema). Por lo tanto en este caso hace falta un cambio de signo a toda la función.
- 0.1 punto La función heurística permitiría minimizar indirectamente el número de días en los que se celebra la competición, pero no tiene en cuenta la minimización del número de días para cada categoría.
- 0.2 puntos Dado que tanto la codificación propuesta junto con el generador de la población inicial pueden representar y generar no soluciones, y el operador también tienen el potencial de generar no-soluciones, es importante añadir al heurístico penalizaciones proporcionales para cada tipo de no-soluciones, y pesos para poder equilibrar la influencia de cada una de ellas.

Puntos extra:

- 0.2 puntos Si asumimos que el número de franjas totales es mayor que el número de partidas por colocar ($FT > PT$) y que la codificación propuesta en el enunciado tiene tamaño $FT \cdot \lceil \log_2(PT+1) \rceil$, en este problema se podría reducir el tamaño de la representación teniendo un vector de partidas, donde cada partida tiene asignado el identificador (en binario) de la franja en la que se disputará, dando una representación de tamaño $PT \cdot \lceil \log_2(FT + 1) \rceil$. Esta codificación sigue permitiendo codificar todas las soluciones y también no-soluciones, pero menos que antes, y además en una solución no tendremos codificaciones "vacías" (todo ceros). Otra posible codificación mejor sería tener una matriz binaria de *días* $\times$ *franjas* $\times$ *partidas* con cada casilla un booleano que si está a 1 indica que la partida i -ésima está asignada a la franja j -ésima del n -ésimo día. Esta representación posiblemente tenga un tamaño mayor y numerosos ceros, pero evitaría *NS1* y permitiría encontrar operadores de cruce y mutación que eviten generar las no-soluciones *NS2* y *NS3* (dada una partida i solo habrá una casilla con valor 1 para todo día y toda franja). Eso si, es difícil encontrar una representación mejor donde se pueda controlar, a nivel de codificación binaria, las restricciones sobre categorías y rondas que puedan generar las no-soluciones *NS4*, *NS5* y *NS6*.
- 0.1 punto El operador de cruce, al no cortar dentro de la codificación de los identificadores de partida, realiza una exploración más lenta del espacio de soluciones / no-soluciones.
- 0.2 puntos Con la primera representación alternativa (donde el vector de bits codifica las franjas donde se coloca cada una de las partidas), una mezcla de cruce y mutación cubriría mejor el espacio, y al alterar las codificaciones estaría "moviendo" partidas (aunque habría casos en los que movería una partida a una franja ya ocupada, a una franja que no existe) y seguirían generando no-soluciones *NS4*, *NS5* y *NS6*. Con la segunda representación alternativa propuesta (la matriz binaria de *días* $\times$ *franjas* $\times$ *partidas*) una mezcla de operadores de cruce y mutación que

aseguren mantener una partida una sola vez y una sola partida por franja cubrirían mejor el espacio, evitando (*NS1*, *NS2* y *NS3* pero sin poder evitar *NS4*, *NS5* y *NS6*).

Anotación didáctica: El objetivo de la solución planteada era haceros reflexionar que aunque en AG lo ideal es tener representaciones que no permiten codificar no soluciones (para que el generador de soluciones iniciales y los operadores nunca puedan sacarnos del espacio de soluciones), se puede trabajar también con representaciones que permitan no soluciones intentado minimizar las no soluciones en las soluciones iniciales y teniendo heurísticos que promuevan las buenas soluciones por encima de las no-soluciones.

Como los operadores genéticos no tienen función de aplicabilidad, existe la tentación de restringirlos para que no puedan generar codificaciones erróneas. Pero en este problema se ha llevado al extremo (el operador de corte no puede generar nuevos identificadores de partida, el operador de mutación lo quitamos para que no genere no soluciones) y el resultado es que no cubrimos correctamente el espacio de soluciones/no soluciones.

Algunos alumnos/as añadían como crítica que "la representación no tiene en cuenta las restricciones". Esto no tiene sentido. Una representación no "tiene en cuenta restricciones", lo que puede hacer es representar soluciones que cumplan o no ciertas restricciones, y pueden (por su forma, estructura o rango de valores) asegurar que alguna restricción se cumpla siempre.

Algunos alumnos quejaban que los operadores no tenían en cuenta las restricciones, pero los operadores genéticos son así: se aplican directamente sobre la codificación sin decodificarla para ver si el resultado es válido o no. Eso se hace una vez en cada ciclo a la vez que intenta evaluar la calidad de la solución generada tras el cruce y mutación.

Algunos alumnos/as han sugerido penalizar con coste infinito "las no-soluciones. Esto no es una buena idea ni siquiera en AG, ya que no guía al algoritmo cuando se encuentra en el espacio de no-soluciones.

2. (4 puntos) Los organizadores de la 080 Barcelona Fashion Week nos piden un sistema inteligente que ayude a planificar los desfiles de la próxima edición. El sistema ha de organizar 24 desfiles para 24 diseñadores, 6 por día en un total de 4 días, sabiendo que algunos diseñadores se niegan a desfilan el mismo día que algún otro. Tenemos una bolsa de modelos que podemos contratar para desfilan. Se han de asignar modelos a los desfiles (m por desfile) sin que la misma persona desfile más de tres veces en total durante la semana, y tampoco puede desfilan más de dos veces el mismo día. Cada diseñador tiene también unas restricciones de altura y peso para sus modelos. Los organizadores de la pasarela han de pagar a los modelos y cada uno tiene su tarifa por participar en un desfile. Obviamente, los organizadores prefieren minimizar el coste. Tras un análisis inicial del problema un compañero de nuestra empresa nos plantea dos estrategias distintas para resolverlo.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (estado o solución inicial, operadores, función heurística,...). Para cada una de ellas se ha de comentar los diferentes elementos de la propuesta, analizando si la técnica escogida es adecuada para este problema, si cada uno de sus elementos son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allí donde sea necesario. Justifica tu respuesta.

- a) Usar el algoritmo de A*. El estado es una asignación parcial de diseñadores y modelos a desfiles, empezando desde ninguna asignación (ni modelos ni diseñadores). Se definen dos operadores: **añadir_diseñador** a un desfile concreto, que comprueba que el diseñador no se haya añadido antes a algún desfile y que en caso de haber ya otros diseñadores asignados en desfiles de ese día, no haya incompatibilidad con esos diseñadores, el coste de este operador es siempre 1; y **añadir_modelo** a un desfile concreto, que comprueba que el desfile tenga ya diseñador asignado, que no haya más de m modelos en el desfile y que se cumplan las restricciones del diseñador sobre los modelos. El coste de este operador es la tarifa del modelo asignado. Como función heurística usamos el número de diseñadores que falta incluir en los desfiles más el número de modelos que falta incluir en los desfiles multiplicado por la tarifa del modelo más caro.

A continuación se muestra una posible solución, puntuada por apartados (sobre 2,6 puntos). Más abajo se muestra un razonamiento más completo pero no representativo de lo que se pide en el examen, como anotaciones didácticas.

- 0.4 puntos A* no es el mejor algoritmo porque su eficiencia es baja a cambio de obtener el óptimo global que no es necesario en este caso (otros razonamientos son también válidos). Empezar desde vacío es correcto ya que permite explorar bien el espacio de soluciones.
- 0.8 puntos Los operadores tienen precondiciones casi correctas: añadir modelo requiere comprobar que no se añada de más (>2 por día, >3 en total). La ramificación es de $Dis * Desf = 24^2$ y $modelos * desf$. Los costes de los operadores están en unidades diferentes, y el coste de añadir diseñador no es algo que se pida minimizar, pero es necesario en A* para evitar problemas.
- 0.4 puntos El heurístico no es admisible, pero se arregla rápidamente si cambiamos 'modelo más caro' por 'modelo más barato'.
- 0.6 puntos El heurístico parece muy poco informado. Lo único que discrimina los estados es 'el número de modelos/diseñadores asignados'. Explorará demasiado el espacio de estados, ya que no distingue entre añadir modelos, sólo cuenta cuántos hay asignados.

Puntos extra:

- 0.4 puntos Hay muchísimos caminos para llegar a un mismo estado. Esto explota el espacio de búsqueda. En su lugar, podemos forzar a asignar en orden (primero el diseñador del primer día, primera hora, luego el segundo, etc. y luego el primer modelo del primer desfile...) la búsqueda es en forma de árbol y es más sencilla.

Anotación didáctica no representativa de lo que se espera en una solución: Si bien A* se puede modelar para este problema, y es suficiente el analizar la propuesta y arreglar los errores sencillos (1,6/2 puntos), la dificultad principal está en ver que no es un algoritmo adecuado, y esto se refleja en los 'parches' utilizados.

Tenemos que añadir un coste inexistente (al añadir diseñador) porque hay un elemento que no tiene coste real y se debe decidir. Además, añadir modelos no añade información real sobre la distancia esperada al objetivo, que se calcula de forma muy cándida (si se considera el modelo más barato,

si se considera el más caro directamente no es admisible). El algoritmo explora básicamente como un BFS.

Esto se ve cuando se considera $f = g + h$. Al principio, éste vale: $g + [h] = 0 + [24 + B * 24 * m]$, donde B es el coste del modelo más barato, y m el número de modelos por desfile. **Toda** asignación de diseñadores vale lo mismo (pero desplazando unidades del 24 en la h a la g). Es más, cualquier estado que solo use el modelo más barato, también (moviendo B de uno en uno de h a g). La cantidad de estados que representa esto es exponencial. El algoritmo explora **todos** estos estados antes de intentar añadir al segundo modelo:

- 1) Primero añade un diseñador, que es el único operador que puede hacer.
- 2) Luego añade el modelo B a dicho desfile si puede (si no, añade otro diseñador). Esto lo hace antes de añadir otro diseñador, porque ante misma f , prioriza el que tenga menor h .
- 3) El algoritmo continúa alternando añadir diseñador con usar el modelo B . Cuando éste se ha usado el máximo de veces (3), el algoritmo continúa hasta añadir el último diseñador.
- 4) Cuando ya no puede añadir diseñadores ni añadir el modelo B más veces, el algoritmo empieza haciendo backtrack de los diseñadores, probando toda posible combinación de diseñador-desfile; y luego cambiando a qué desfile está B asignado.

Solamente cuando haya terminado con toda esta combinatoria, procederá a usar el segundo modelo por orden de coste ($B2$, y probar de nuevo toda la combinatoria modelo B con una instancia de $B2$, etc.

Todo esto se resume en una búsqueda exponencial muy innecesaria, y más ineficiente que un BFS relativamente bien programado.

Esto se puede arreglar en la heurística: en lugar de contar el más barato por el número de restantes, si K es el número de modelos por asignar ($K = 24 * m - \text{asignaciones}$), se puede coger el top- K de los precios más baratos que siguen disponibles (p.ej. si el modelo más barato se ha asignado 2 veces y el 2º más barato solo una, $B + 2 * B2 + \dots$). Este es el precio mínimo asegurado (no puede haber una asignación menos costosa). De esta manera, todos los mejores modelos se pueden asignar, todos los estados tienen la misma f , y se explora toda combinatoria óptima primero (donde combinación óptima significa que todos los $K/3$ modelos más baratos se han asignado 3 veces cada uno, completando el evento). Si no es posible asignar solo los mejores modelos, solo entonces el algoritmo considerará intercambiar el modelo más caro de los más baratos por el más barato de los caros (no considerados).

Además, para evitar todas estas búsquedas de permutaciones, se puede hacer lo que vimos con A^* los primeros días de clase: forzar a que la asignación sea en orden, ya que no hay diferencia sobre en qué orden se aplican los operadores. Si no hacemos esto, en un estado donde hay K cosas asignadas, hay $O(K!)$ maneras de llegar a ella (permutaciones sobre en qué orden se añaden los elementos).

- b) Usar un algoritmo de constraint programming. Se definen tres variables por cada uno de los modelos que representan los tres posibles desfiles que se le pueden asignar a cada modelo m ($\text{desfile}[m][1]$, $\text{desfile}[m][2]$, $\text{desfile}[m][3]$) y una variable por cada uno de los 24 diseñadores d ($\text{asignado}[d]$), y una variable P . El dominio de cada una de esas variables sería el identificador del desfile a asignar (un número entero entre el 1 y el 24) y los números reales para P . Como restricciones binarias habría: **restricciones** de desigualdad entre todo par de variables desfile de un mismo modelo m ($\text{desfile}[m][i] \neq \text{desfile}[m][j]$) de modo que tengan asignado un desfile diferente; **restricciones** de desigualdad entre todo par de variables $\text{asignado}[d1]$ y $\text{asignado}[d2]$ (cada diseñador ha de tener asignado un desfile diferente); **restricciones** para cada modelo m que no cumple las restricciones de altura y peso de un diseñador d , estableciendo desigualdad entre la variable diseador_d y las tres variables $\text{desfile}[m][i]$ (el modelo m no puede coincidir en ninguno de sus desfiles asignados con el diseñador d). Por último, la variable P debe minimizarse, y tiene una **restricción** n-aria con todas las variables de desfile tal que P valga la suma de, para cada desfile, la tarifa del modelo asignado.

A continuación se muestra una posible solución, puntuada por apartados (sobre 2 puntos). Más abajo se muestra un razonamiento más completo pero no representativo de lo que se pide en el examen, como anotaciones didácticas, y con hasta 0,6 puntos extra.

0.6 puntos Optimización de restricciones puede venir bien para un problema donde hay que hacer una asignación con restricciones y se debe optimizar (otros razonamientos son válidos). Este pro-

blema está mal planteado, sin embargo, porque no permite dejar modelos sin asignar: faltaría permitir la asignación vacía en el dominio.

1 punto Las restricciones que hay son correctas, pero falta:

- No puede darse que un modelo esté asignado a >2 desfiles en un día. Son M restricciones trinarias (M siendo el número de modelos disponibles).
- Cada desfile debe tener m modelos asignados (si no, el problema es trivial: no asignes ningún modelo). Es una n -aria de $3 * M$ variables.
- Dos diseñadores que se odian no pueden desfilan el mismo día. Son muchas restricciones binarias (grafo completo entre diseñadores).

0.4 puntos La representación es terrible porque con un M no demasiado pequeño, se requieren de muchísimas variables que al final no serán asignadas. Es mejor una representación alternativa, donde en lugar de las variables $desfile[m][1..3] = [0, 24]$, se usa $modelo[i][j] = [1..M]$: Para cada uno de los 24 desfiles, usamos m variables cuyo contenido es el modelo que desfila. Esto es drásticamente más pequeño y manejable; elimina la n -aria tan grande, pero modifica un poco la cardinalidad de otras restricciones.

Se han dado puntos extra por [0.2] mencionar como la restricción sobre P que a priori es n -aria se puede descomponer en restricciones trinarias añadiendo variables, y por [0.4] remarcar que una representación alternativa como la propuesta tiene muchas simetrías (para un mismo desfile, hay $m!$ formas de representar la misma asignación de modelos).

Anotación didáctica: este es un problema bastante representativo de constraint programming. Los dos puntos más importantes de éste son los dos problemas de sentido común: que no se permite dejar modelos sin asignar, y que en cualquier caso, no se asegura de que los desfiles tengan exactamente m modelos. Hay un par más de restricciones que faltan, pero estas dos son lo más importante dado que el modelo es inútil sin éstas.

Por otra parte, para demostrar maestría sobre la técnica, un modelador debería percatarse de que la representación es muy ineficiente: con una bolsa de modelos grande (como se puede asumir, o al menos una no mínima) el número de variables es muy superior al necesario, y girar la representación es mucho más eficiente.

Las notas se publicarán antes del día **21 de noviembre**.